

Reviewer Pack — Algebraic Curve Genus and Communication Complexity Lower Bou...

Entry e7614c62651c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 09:12:58 UTC

Algebraic Curve Genus and Communication Complexity Lower Bound

Entry ID: e7614c62651c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 09:12:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any Boolean function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, the genus g of the algebraic curve defined by its communication matrix satisfies $g \geq \log^2(\text{Comm}(f))$

Rationale (proposer's reasoning):

Algebraic curves encode structural constraints on polynomial representations of communication matrices. The genus, a topological invariant, may capture inherent complexity barriers through its relation to polynomial degree and ramification, offering a new lens to analyze communication complexity lower bounds.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4e5447106e5e2c64

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                continue
            clauses.append(clause)
        return clauses

    def communication_matrix(cnf):
        m = len(cnf)
        n = max(abs(x) for x in cnf[0])
        A = [[0] * (2**n) for _ in range(2**n)]
        for i, clause in enumerate(cnf):
            for j in range(len(clause)):
                if clause[j] > 0:
                    A[i][j] += 1
                else:
                    A[i][j] -= 1
        return A

    def determinant(A):
        n = len(A)
        det = 0
        if n == 1:
            return A[0][0]
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1)**j * A[0][j] * determinant(submatrix)
        return det

    def genus_formula(determinant, n):
        # Simplified genus formula for plane curves
        return 1 + (n - 1) // 2
```

```

def comm(f):
    # Deterministic protocol to measure communication complexity
    return len(f)

n = random.randint(5, 40)
cnf = generate_cnf(n)
A = communication_matrix(cnf)
det = determinant(A)
g = genus_formula(det, n)
Comm_f = comm(cnf)

if g < math.log2(Comm_f) ** 2:
    return {
        "metric_name": "genus",
        "metric_value": g,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Counterexample for n={n}: genus < log^2(Comm(f))"
    }

return {
    "metric_name": "genus",
    "metric_value": g,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        from sympy.ntheory import primerange
        seeds = list(primerange(2, 100))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and debug crash cause

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106914
2	propose	ollama_remote	qwen3:8b	0	0	109677
3	propose	ollama_remote	qwen3:8b	0	0	113229
4	propose	ollama_remote	qwen3:8b	0	0	155390
5	preregistration	ollama_remote	qwen3:8b	0	0	26230
6	novelty	ollama_remote	qwen3:8b	0	0	21415
7	novelty	ollama_remote	qwen3:8b	0	0	15753
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23733
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13808
10	judge	ollama_remote	qwen3:8b	0	0	114529

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 700678 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e7614c62651c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e7614c62651c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e7614c62651c.tar.gz` (if generated)